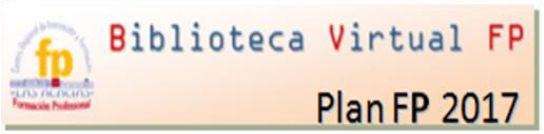
Hibernate

Tecnologías avanzadas para
el desarrollo con Java
IFC03CM17



Introducción

- Hibernate es:
  - Un framewrok de persistencia.
  - Una herramienta ORM (*Object Relational Mapping* – Mapeo Objeto Relacional).
- Crea clases en un lenguaje OO, como Java, asociadas a tablas de una BD relacional.
- Permite trabajar a nivel de aplicación con objetos, sin preocuparse de construir instrucciones DML de SQL.
- Como toda capa de abstracción intermedia:
  - Facilita el trabajo
  - Incide negativamente en el rendimiento

Introducción II

- Hibernate implementa la especificación JPA (*Java Persistence API* – JSR 338), y añade elementos propios. Es un superconjunto de JPA.
- Existen otros *frameworks* que implementan JPA:
  - ObjectDB
  - EclipseLink
  - OpenJPA

Planes de estudio de Informática

- Se imparte en DAM2 → Acceso a datos
- Se debería introducir en DAW2 → Desarrollo en entorno servidor

Cómo trabajar con *Hibernate* I

- 2 formas de ingeniería:
  - **Directa**: a partir de los POJOs, se construye el modelo físico de la BD.
  - **Inversa**: a partir del modelo de datos, se generan los POJOs anotados y los archivos de configuración.
- **Hibernate no está concebido para construir un esquema físico completo y detallado a partir de POJOs anotados.**

Cómo trabajar con *Hibernate* II

- Lo más común al trabajar con Hibernate es:
  1. Partir de un modelo de datos existente
  2. Mediante ingeniería inversa, y asistido por alguna herramienta de generación automática de código (*Hibernate Tools*), generar los archivos de configuración, los POJOs, …
  3. Empezar a desarrollar la aplicación con los POJOs generados como entidades de negocio, utilizando las clases y métodos que ofrece *Hibernate* para persistir/recuperar objetos de la BD

Archivos de configuración y generados

- hibernate.cfg.xml

  Archivo principal de configuración. Documento XML que configura los aspectos principales de la conexión a la BD.
- hibernate.reveng.xml

  Archivo de ingeniería inversa. Documento XML.
- Mapeo

  Documentos XML o anotaciones dentro de los POJOs, que describen las relaciones entre las tablas de la base de datos y los POJOs
- POJOs

  Clases Java, representativas de las entidades de negocio, generadas a partir de la base de datos subyacente

hibernate.cfg.xml
Archivo principal de configuración I

Propiedad	Uso
`hibernate.connection.driver_class`	Driver de la conexión (MySQL, Oracle…)
`hibernate.connection.url`	URL de la conexión
`hibernate.connection.username`	Nombre del usuario de la BD
`hibernate.connection.password`	Clave del usuario de la BD
`hibernate.connection.pool_size`	Tamaño del pool de conexiones a la BD
`hibernate.dialect`	SGBD subyacente: MySQL5, Oracle9i, SQLServer
`hibernate.show_sql`	Mostrar el SQL que generan los accesos
`hibernate.use_sql_comments`	Se muestran comentarios al SQL

hibernate.cfg.xml
Archivo principal de configuración II

Propiedad	Uso
`hibernate.hbm2ddl.auto`	Indica si se reconstruye/actualiza el esquema en cada uso (`create` vs. `update`)
`hibernate.cache.use_second_level_cache`	Uso o no del segundo nivel de caché
`hibernate.cache.provider_class`	Clase que gestionará la caché de segundo nivel

hibernate.cfg.xml

```
<?xml version="1.0" encoding="utf-8"?> <!DOCTYPE hibernate-
configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD
3.0//EN" "http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
  <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriv
er</property>
  <property
name="hibernate.connection.url">jdbc:oracle:thin:@127.0.0.1:1521:MKYON
G</property>
  <property name="hibernate.connection.username">mkyong</property>
  <property name="hibernate.connection.password">password</property>
  <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prope
rty>
  <property name="hibernate.default_schema">MKYONG</property>
 </session-factory>
</hibernate-configuration>
```

Archivo hibernate.properties

- Se puede crear un archivo externo de propiedades.

```
hibernate.connection.driver_class=
    org.postgresql.Driver
hibernate.connection.url=
    jdbc:mysql://localhost:3306/mydatabase
hibernate.connection.username=root
hibernate.connection.password=root
// c3p0 -> pool de conexiones integrado en Hibernate
hibernate.c3p0.min_size= 5
hibernate.c3p0.max_size=20
hibernate.c3p0.timeout=1800
hibernate.c3p0.max_statements=50
hibernate.dialect =
    org.hibernate.dialect.MySQLDialect
```

Mapeo en hibernate.cfg.xml

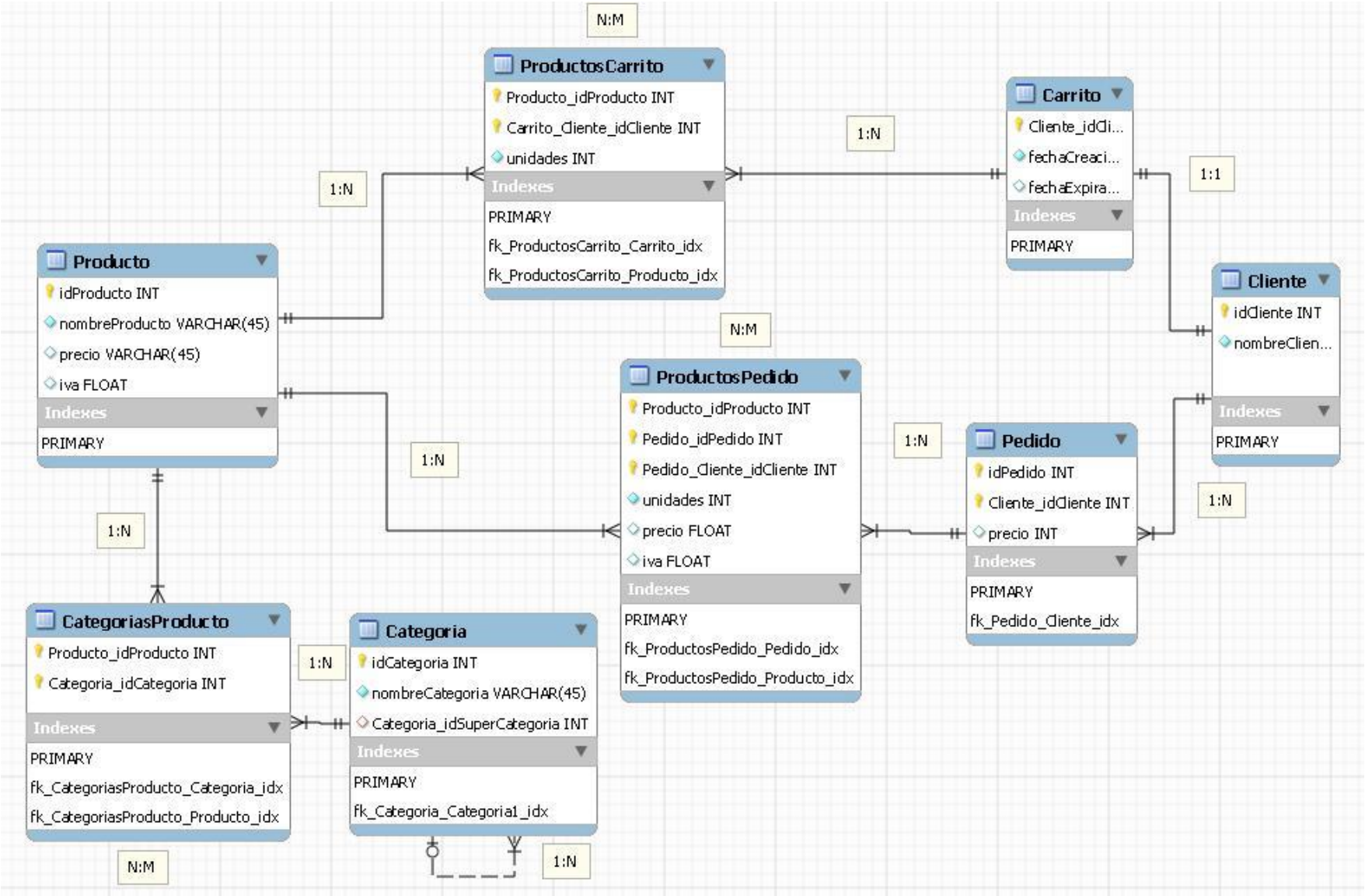
- En `hibernate.cfg.xml` hay que añadir los mapeos, que pueden ser:
  - Recursos: archivos XML (tiende al desuso):

```
<mapping resource=
      "generadoHibernate/Emp.hbm.xml"/>
```

  - Clases anotadas (en auge):

```
<mapping class="generadoHibernate.Emp"
/>
```

Ejemplo de modelo – E/R



Ejemplo de modelo - Mapeo

Tabla de la BD	Clase(s) de la aplicación (POJOs)
Carrito	Carrito.java
Categoria	Categoria.java
Cliente	Cliente.java
Pedido	Pedido.java PedidoId.java
ProductosPedido	ProductosPedido.java ProductosPedidoId.java
ProductosCarrito	Productoscarrito.java ProductoscarritoId.java
Producto	Producto.java
CategoriasProducto	---

Tabla	Ej	Clase	Ej
Maestra	Cliente	Una clase	Cliente.java
Intermedia de una relación N:M, sin atributos	CategoriasProducto	No aparece ninguna clase	
Intermedia de una relación N:M, con atributos	ProductosCarrito	Dos clases: - Una contiene un objeto identificador y los atributos - Otra es la clase del objeto identificador, compuesta de las claves ajenas a las tablas circundantes	ProductosCarrito.java ProductosCarritoId.java
Detalle, con clave primaria compuesta de más de un campo	Pedido	Dos clases: - Una contiene un objeto identificador y los atributos - Otra es la clase del objeto identificador, compuesta de las claves ajenas a las tablas circundantes	Pedido.java PedidoId.java

Mapeo de claves primarias multicampo

- Cuando en una tabla T, la PK está compuesta por más de un campo, *Hibernate* genera:
  - una clase T.java para mapear la tabla T
  - una clase auxiliar TId.java cuyos atributos son los que forman la PK de la tabla
- La clase T.java contendrá un atributo de la clase TId.java, que representará el id de los objetos de la clase T.

POJOs

- **P**lain **O**ld **J**ava **O**bject
- Son generados por *Hibernate*.
- Clases típicas de un lenguaje OO
- Atributos: mapeados a campos de la BD
- Métodos:
  - Constructor(es)
  - Getters o de consulta
  - Setters o de modificación

Archivos de mapeo (al desuso)

- Es un documento XML.
- El elemento raíz es `<hibernate-mapping>`.
- Elementos:
  - `<class>`
  - `<meta>`
  - `<id>`
  - `<generator>`
  - `<property>`

`<class>`

- Permite definir mapeos entre clases java (POJOs) y tablas de la BD.
- Atributos:
  - `name`: nombre de la clase Java.
  - `table`: nombre de la tabla de la BD.

`<id>`

- Mapea los atributos que forman el identificador único de la clase con la clave primaria de la tabla.
- Atributos:
  - `name`: nombre del atributo(s) de la clase Java.
  - `column`: nombre del campo(s) de la tabla.
  - `type`: tipo de datos de mapeo de Hibernate, para convertir el del atributo en el de la tabla SQL.

`<property>`

- Mapea los atributos de la clase con campos de la tabla.
- Atributos:
  - `name`: nombre del atributo(s) de la clase Java.
  - `column`: nombre del campo(s) de la tabla.
  - `type`: tipo de datos de mapeo de Hibernate, para convertir el del atributo en el de la tabla SQL.

Anotaciones

- Archivos jar:

  `hibernate-commons-annotations.jar`

  `ejb3-persistence.jar`

  `hibernate-annotations.jar`

- Se recomienda, por motivos de compatibilidad con JPA, su importación del paquete

  `javax.persistence`

- Las anotaciones específicas de *Hibernate* (no estándar de JPA) se importan del paquete:

  `org.hibernate.annotations`

Anotaciones principales

- **@Entity**

  Indica que una clase es un *bean* de Entidad, ie, la clase tendrá, al menos, un constructor sin argumentos con acceso a nivel de paquete.

- **@Table**

  Especifica una tabla donde almacenar datos. El atributo *name* es el nombre de la tabla. Si no se especifica, Hibernate tomará como nombre de la tabla el nombre de la clase.

Anotaciones principales

- **@Id**

  Se usa para indicar la propiedad(es) identificadoras del *bean* Entidad.

  La ubicación de esta anotación determina la estrategia de acceso por defecto que Hibernate usa para el mapeo.

  Si la anotación se ubica sobre el atributo, se usará **acceso por campo**.

  Si la anotación se ubica sobre el método *getter* del atributo, entonces se usará el **acceso por propiedad**.

Anotaciones principales

- **@Column**

  Información sobre un campo de una tabla mapeada en la base de datos: nombre, catálogo…

- **@GeneratedValue**

  La BD genera el identificador (ej. campo auto-numérico)

- **@OneToOne , @ManyToOne, @OneToMany y @ManyToMany**

  Tipo de relación entre tablas

Anotaciones I

Anotación	Atributo	Significado
`@Entity`		Entidad *Hibernate*
	`name`	Nombre de la entidad y de la tabla subyacente
`@Table`		Tabla de la base de datos
	`name`	Nombre de la tabla en la BD, no de la entidad
	`catalog`	Catálogo (esquema / usuario) de la BD
	`uniqueConstraints`	Lista de elementos `@UniqueConstraint`, en los que se definen la restricciones de unicidad existentes en la tabla
`@Id`		Campo que forma parte de la PK
`@UniqueConstraint`		Define una restricción de unicidad sobre una tabla
	`columnNames`	Columna(s) a la(s) que afecta la restricción
	`name`	Nombre de la restricción de unicidad

Anotaciones II

Anotación	Atributo	Significado
@Column		Campo (columna) de una tabla de la BD
	name	Nombre del campo
	unique	Existe una restricción de unicidad
	nullable	¿Puede ser nulo?
	updatable	Se puede modificar su valor
	insertable	Se puede asignar un valor
	length	Longitud de la columna
	precision	Número de posiciones (dígitos, caracteres)
	scale	Número de dígitos decimales
	columnDefinition	Tipo de datos en el SGBD subyacente
@Basic		Atributo que se persistirá. No es obligatorio explicitarlo
	fetch	Forma de recuperarlo: LAZY o EAGER
	optional	

Anotaciones III

Anotación	Atributo	Significado
`@Temporal`		Atributo de fecha y/u hora
	`TemporalType`	Enumerado: `DATE`, `TIME`, `TIMESTAMP`
`@Lob`		Atributo que contiene un objeto grande. Corresponde con el tipo de datos LOB (*Large OBject*), existente en algunos SGBDs
`@MapsId`		
`@EmbeddedId`		Indica que un atributo, que es clave, es un objeto de una clase embebida. Es frecuente en las PK compuestas por más de un campo
`@Embedded`		Indica que un atributo es un objeto de una clase embebida, el cual tiene un valor propio para cada clase contenedora.
`@Embeddable`		Indica que una clase será embebida (dependiente) dentro de otra
`@Access`		Indica el modo de acceso a los atributos: **`PROPERTY`** (a través del *getter*) o **`FIELD`** (al atributo directamente)

Anotaciones IV

Anotación	Atributo	Significado
`@AttributeOverrides`		Indica que se va a sobrescribir uno o más atributos
`@AttributeOverride`		Se modifica la columna a la que se asocia un atributo
	`name`	Nombre del atributo
	`column`	Nombre de la columna
`@AssociationOverride`		
`@GeneratedValue`		Se genera automáticamente una clave para el campo (usa secuencia interna de Hibernate)
	`strategy`	Estrategia para generar el valor: `AUTO`, `IDENTITY`, `SEQUENCE`, `TABLE`. Cada opción depende de lo que ofrezca el SGBD subyacente. Se recomienda `AUTO`, para que Hibernate decida la mejor opción.
	`generator`	

Anotaciones V

Anotación	Atributo	Significado
`@OneToOne`		Relación 1 a 1 entre dos objetos entidad
`@OneToMany`		Relación 1:N
	`fetch`	Modo de recuperación
	`mappedBy`	Indica el nombre del atributo en la entidad propietaria. Aparece en el lado no propietario de la relación
	`cascade`	Cómo proceder con los objetos relacionados. Las opciones son: `ALL`, `DETACH`, `MERGE`, `PERSIST`, `REFRESH` y `REMOVE`
	`orphanRemoval`	
	`targetEntity`	
`@ManyToOne`		Relación N:1
	`fetch`	Modo de recuperación
`@ManyToMany`		Relación N:M
	`mappedBy`	Indica el nombre del atributo en la entidad propietaria. Aparece en el lado no propietario de la relación

Anotaciones VI

Anotación	Atributo	Significado
`@JoinColumns`		Lista de columnas de vinculación
`@JoinColumn`		Campo de vinculación de una relación 1:1 o N:1
	`name`	Nombre del campo que será la clave ajena en la tabla origen
	`referencedColumnName`	Nombre del campo en la tabla referenciada
`@PrimaryKeyJoinColumn`		Columna que es clave primaria de una tabla y además clave ajena que referencia a otra. Es frecuente que sea: - la clave primaria de una tabla resultante de una relación de dependencia en identificación - la clave primaria de una tabla resultante de una sub-entidad en una jerarquía
	`name`	Nombre de la columna
	`referencedColumnName`	Nombre del campo que es clave primaria en la tabla referenciada

Anotaciones VII

Anotación	Atributo	Significado
@JoinTable		Tabla intermedia de una relación (ej. de una clase contenedora con una embebida) (ej. en una relación N:M)
	name	Nombre de la tabla
	joinColumns	Columna(s) de vinculación
	catalog	
	inverseJoinColumns	Columna(s) de vinculación inversa
	schema	
	uniqueConstraints	
@ElementCollection		Atributo que es una colección de objetos pertenecientes a una clase embebible
	fetch	Estrategia de recuperación (LAZY por defecto)
	targetClass	
@CollectionTable		

Anotaciones VIII

Anotación	Atributo	Significado
`@Inheritance`		Indica que de una clase aparecerán subclases (herencia)
	`strategy`	Especifica cómo se implementará la herencia: `SINGLE_TABLE`, `JOINED`, `TABLE_PER_CLASS`
`@DiscriminatorColumn`		Describe cuál será la columna discriminante en la tabla subyacente. Tiene sentido para `SINGLE_TABLE`
	`name`	Nombre de la columna
	`discriminatorType`	Tipo de datos de la columna: `CHAR`, `INTEGER`, `STRING`
`@DiscriminatorValue`		Valor que tomará la columna discriminante para una determinada clase. Tiene sentido para `SINGLE_TABLE`
`@MappedSuperclass`		Para crear una superclase que sirva

Anotaciones IX

Anotación	Atributo	Significado
`@NamedQuery`		Define una consulta HQL con nombre. Se definen en el POJO
	`name`	Indica el nombre de la consulta
	`query`	Código HQL de la consulta
`@NamedNativeQuery`		Define una consulta SQL con nombre. Se definen en el POJO
	`name`	Indica el nombre de la consulta
	`query`	Código SQL de la consulta
	`resultClass`	Clase de la que será el resultado
	`resultSetMapping`	
`@NamedQueries`		Define una lista de `@NamedQuery` en un POJO
`@NamedNativeQueries`		Define una lista de `@NamedNativeQuery` en un POJO

Anotaciones X

Anotación	Atributo	Significado
`@NamedStoredProcedureQuery`		Consulta con nombre a un procedimiento almacenado
	`name`	Nombre de la consulta
	`procedureName`	Nombre del procedimiento almacenado
	`resultClasses`	Clase(s) resultante(s) de la llamada al procedimiento almacenado
	`parameters`	Parámetro(s) a pasar/devolver al/del procedimiento almacenado
`@StoredProcedureParameter`		Parámetro de un procedimiento almacenado
	`name`	Nombre del parámetro
	`type`	Tipo del parámetro
	`mode`	De entrada (`IN`) o salida (`OUT`)

Anotaciones XI

Anotación	Atributo	Significado
`@CollectionTable`		
	`name`	Nombre de la tabla
	`joinColumns`	
`@OrderColumn`		
`@SecondaryTables`		Define una lista de elementos `@SecondaryTable`
`@SecondaryTable`		
	`name`	
	`pkJoinColumns`	
	`uniqueConstraints`	
`@NaturalId`		

Anotaciones XII

Anotación	Atributo	Significado
@MapKeyColumn		
@MapKeyEnumereted		
@MapKeyTemporal		
@MapKeyJoinColumn		
@MapKeyJoinColumns		
@Cacheable		
@Cache		
	usage	
	region	
	include	
@SqlResultSetMapping		

Anotaciones XIII

Anotación	Atributo	Significado
`@EntityResult`		
`@FieldResult`		
	`name`	
	`column`	
`@ColumnResult`		
	`name`	
`@Inmutable`		Establece una entidad o una colección es inmutable
`@Persister`		
`@BatchSize`		
`@Formula`		
`@Index`		Define un índice sobre una(s) columna(s)
	`name`	Nombre del índice

Anotaciones XIV

Anotación	Atributo	Significado
`@Parent`		Permite vincular un objeto de una clase embebida con su objeto contenedor
`@Generated`		
`@Target`		
`@OnDelete`		
	`action`	
`@ForeignKey`		Permite definir una clave ajena sobre un campo
	`name`	Nombre de la clave ajena
`@Sort`		
`@SQLInsert`		
`@SQLUpdate`		
`@SQLDelete`		
`@SQLDeleteAll`		
`@Loader`		

Anotaciones no JPA estándar Específicas de *Hibernate* I

- Paquete: `org.hibernate.annotations`

Anotación	Atributo	Significado
`@CollectionId`		Indica que un atributo colección es clave
	`columns`	La(s) columna(s) que forma(n) la clave primaria
	`generator`	Cómo se genera el valor de la clave primaria
	`type`	
`@GenericGenerator`		
	`name`	
	`Strategy`	
`@Type`		Indica el tipo de datos de una columna usada para ser clave
	`type`	El tipo de datos (ej. `long`)
`@TypeDef`		

Anotaciones no JPA estándar Específicas de *Hibernate* II

Anotación	Atributo	Significado
`@NotFound`		Previene que se lancen excepciones cuando no hay datos en una relación ???
	`action`	Qué se hace si no hay correspondencia. Tiene que valer: `NotFoundAction.IGNORE`
`@Entity`		Propiedades de la entidad
	`selectBeforeUpdate`	Indica a Hibernate que antes de ejecutar un `session.update`, compruebe si el objeto se ha modificado, realizando un `select` Útil si no se suele modificar el objeto entre sesiones, sino, penaliza con un `select` innecesario
`@ForeignKey`		Define una FK con nombre
	`name`	Nombre de la FK
	`inverseName`	

Ejemplo de Anotaciones

- @Table(name = "emp", catalog = "scott")
- @Column(name = "empno", unique ="true", nullable ="false", precision ="4", scale ="0")
- @ManyToOne(fetch = FetchType.LAZY)
- @JoinColumn(name = "mgr")
- @Temporal(TemporalType.DATE)
- @OneToMany(fetch = FetchType.LAZY, mappedBy = "emp")

Ejemplo de Anotaciones

- @AssociationOverrides({ @AssociationOverride(name = "pk.stock", joinColumns = @JoinColumn(name = "STOCK_ID")), @AssociationOverride(name = "pk.category", joinColumns = @JoinColumn(name = "CATEGORY_ID")) })
- @SequenceGenerator(name="seq1",sequenceName="HIB_SEQ")
- @GeneratedValue(strategy=SEQUENCE,generator="seq1")

Ejemplo de Anotaciones

- @TableGenerator(name="tablegen", table="ID_TABLE", pkColumnName="ID", valueColumnName="NEXT_ID")
- @EmbeddedId
- @IdClass(Account.AccountPk.class)
- @SecondaryTable(name="CUSTOMER_DETAILS")
- @Table(name="CUSTOMER",uniqueConstraints={ @UniqueConstraint(columnNames="name")}

Mapeos entre entidades

- Unidireccional
- Bidireccional
  - Con mapeo doble a igual nivel
    - La sincronización se mantiene manualmente por programa
    - Los accesos cruzados son eficientes
  - Con mapeo propietario en un sentido y subordinado en el otro
    - La sincronización es automática
    - Los accesos cruzados son poco eficientes (se realizan mediante consultas)

Mapeos bidireccional, propietario en un sentido y subordinado en el otro I

- La entidad **propietaria** de la relación es aquella cuyo POJO se vincula a la tabla detalle de la BD.
- La entidad **subordinada** de la relación es aquella cuyo POJO se vincula a la tabla maestra de la BD. El atributo del POJO que representa la relación se etiquetará como `mappedBy`.

Mapeos bidireccional, propietario en un sentido y subordinado en el otro II

- Ej. Relación 1:N entre Empleados y Departamentos
  - Empleados es la tabla detalle, y su entidad Empleado la propietaria de la relación
  - Departamentos es la tabla maestra, y su entidad Departamento la subordinada en la relación

Operaciones en cascada en relaciones entre entidades

- Existen varios tipos de operaciones en cascada:
  - `PERSIST`: Las operaciones `save()` y `persist()` se propagan
  - `MERGE`: Cuando la entidad propietaria se mezcla (*merge*), las entidades relacionadas se mezclan
  - `REFRESH`:
  - `REMOVE`:
  - `DETACH`:
  - `ALL`: La unión de todas las opciones anteriores

Estrategias en la recuperación de información desde el origen de datos

- Perezosa (`LAZY`)
  - Los datos (colecciones) se recuperan cuando se necesitan. Mientras tanto se trabaja con **objetos proxy**.
  - Es la estrategia de recuperación por defecto.
- Ansiosa (`EAGER`)
  - Los datos (colecciones) se recuperan cuando se invoca el método de lectura del origen de datos

Clases proxy

- Son subclases de clases de negocio.
- Tienen exactamente sus mismos atributos y métodos.
- Según la estrategia de recuperación de datos perezosa (`LAZY`), los *getters* de estas clases que devuelven objetos y colecciones, cuando son invocados, realizan nuevas consultas al origen de datos para recuperar los datos en ese momento, y no antes.
- Su uso es transparente al desarrollador.

Técnicas para recuperar datos de la BD

- Hay varios:
  - Con métodos predefinidos: get, load
  - Con HQL
  - Con el API de *Criteria*

Criteria

- Mecanismo para construir consultas (completas) a base de métodos predefinidos
- Permite generar una consulta básica a la que se le van añadiendo **restricciones**.
- Por defecto, las restricciones se vinculan como operaciones AND.

Restricciones I

Restricción	Equivalente SQL	Uso	Ejemplo
`eq`	`=`	El campo debe ser igual a un valor	`.eq("nombre", "beta")`
`ne`	`<>`	El campo debe ser distinto a un valor	`.ne("nombre", "eta")`
`gt`	`>`	El campo debe ser mayor que un valor	`.gt("id", 5)`
`lt`	`<`	El campo debe ser menor que un valor	`.lt("id", 5)`
`ge`	`>=`	El campo debe ser mayor o igual que un valor	`.ge("id", 5)`
`le`	`<=`	El campo debe ser menor o igual que un valor	`.le("id", 5)`
`like`	`LIKE`	El campo debe ser igual a un patrón (cadenas case-sens)	`.like()`
ilike	`lower() LIKE`	El campo debe ser igual a un patrón (cadenas case-insens)	`.ilike()`

Restricciones II

Restricción	Equivalente SQL	Uso	Ejemplo
in	IN	Serie de valores que puede tomar un campo	.in(new String[] {"alfa", "gamma", "epsilon"})
between	BETWEEN	Rango de valores entre los que debe estar el valor de un campo	.between()
isEmpty		El campo debe ser vacío (cadenas: "", enteros: 0…)	.isEmpty()
isNotEmpty		El campo debe ser no vacío	.isNotEmpty()
isNull	IS NULL	El campo debe ser nulo	.isNull("nombre")
isNotNull	IS NOT NULL	El campo debe ser no nulo	.isNotNull("nombre")
eqOrIsNull	= OR IS NULL	El campo debe ser, bien igual a un valor, bien nulo	.eqOrIsNull()
neOrIsNotNull	<> OR IS NOT NULL	El campo debe ser, bien distinto a un valor, bien no nulo	.eqOrIsNotNull()

Restricciones III

Restricción	Equivalente SQL	Uso	Ejemplo
`or`	`OR`	Restricciones que se deben cumplir una o la otra	`.or()`
`and`	`AND`	Restricciones que se deben cumplir una y la otra	`.and()`
`not`	`NOT`	Restricción que no se debe cumplir	`.not()`
`Conjunction`			`.conjunction()`
`disjunction`			`.disjunction()`
`naturalId`			`.naturalId()`
`sqlRestriction`		Restricción escrita directamente en SQL	`.sqlRestriction("mod(idUsuario,2)=0")`

Restricciones IV

Restricción	Equivalente SQL	Uso	Ejemplo
`sizeEq`			`.sizeEq()`
`sizeNe`			`.sizeNe()`
`sizeGt`			`.sizeGt()`
`sizeLt`			`.sizeLt()`
`sizeGe`			`.sizeGe()`
`sizeLe`			`.sizeLe()`
`allEq`			`.allEq()`
`idEq`			`.idEq()`

Query by Example (QbE)

- Se define un **objeto ejemplo (modelo)**, de la misma clase que el resultado, y se utiliza como patrón que deben cumplir los objetos del resultado
- Algo así como: “devuelve los objetos que sean como este modelo”
- Hay una serie de atributos del objeto modelo que se ignorarán:
  - Identificadores (PK)
  - Asociaciones
  - Propiedades de versión
  - Propiedades de valor nulo (sin valor asignado)

Métodos para el almacenamiento de información en el origen de datos

- `session.save`
- `session.persist`

Métodos para la recuperación de información desde la fuente de datos

- `session.get`
  - Recupera los atributos simples y punteros a las colecciones
- `session.load`

Pasos para usar la API de *Hibernate*

- Crear una factoría de sesiones:

```
SessionFactory factoria =  new
   Configuration()
   .configure().buildSessionFactory;
```

- Crear una sesión a partir de la factoría:

```
Session sesion = factoria.openSession();
```

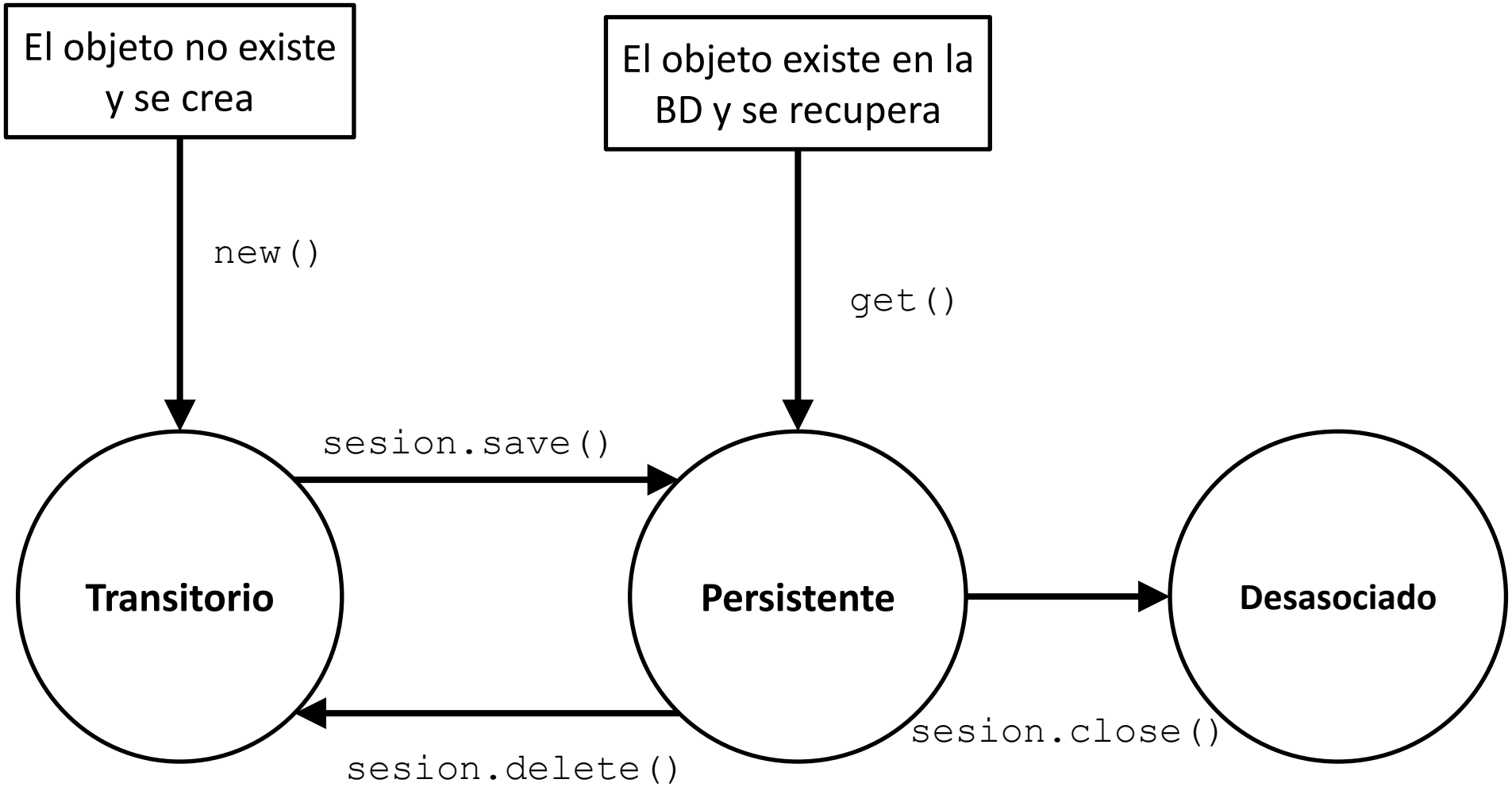
- Usar la sesión para manipular datos:

```
sesion.beginTransaction();
...
sesion.getTransaction().commit();
```

Estados de un objeto

- Existen varios estados en los que se puede encontrar un objeto:
  - **Transitorio** (*Transient*): Nunca se ha guardado en el origen de datos. Hibernate no es consciente de la existencia del objeto.
  - **Persistente** (Persistent): Ya se ha guardado en el origen de datos. Hibernate toma control sobre el objeto.
  - **Desasociado** (*Detached*): Objeto anteriormente persistente, tras cerrarse la sesión. Hibernate abandona el control sobre el objeto.

Cambios en el estado de un objeto



Archivo HibernateUtil.java

- Este archivo gestiona el arranque de *Hibernate* y accede a la `SessionFactory` para obtener un objeto `Session`.
- La clase invoca al método configure() y carga el archivo `hibernate.cfg.xml`, tras lo cual construye la `SessionFactory` para obtener un objeto `Session`.
- En las últimas versiones se desaconseja su uso.

Pasos para modificar correctamente un objeto existente en la BD

- ANTES:
  1. Se abre una sesión.
  2. **Se recupera el objeto de la BD.**
  3. Se cierra la sesión.
- **Se modifica el objeto** (seguramente como consecuencia de una entrada de datos por parte del usuario)
- DESPUÉS:
  1. Se abre una sesión.
  2. Se abre una transacción dentro de la sesión.
  3. **Se persiste el objeto modificado de la BD.**
  4. Se cierra la transacción dentro de la sesión.
  5. Se cierra la sesión.

Clases relevantes

Clase	Uso
`Configuration`	La configuración de la sesión, por defecto extraída de `hibernate.cfg.xml`
`SessionFactory`	Factoría de sesiones
`Session`	Sesión creada desde la factoría de sesiones
`Transaction`	Transacción, necesaria para realizar operaciones de modificación (`INSERT`, `UPDATE`, `DELETE`) en la BD
`Query`	Consulta
`StoredProcedureQuery`	Consulta a un procedimiento almacenado
`Criteria`	Criterios a aplicar a una consulta
`Restrictions`	
`Proyections`	
`Example`	Clase para fijar un objeto que sea modelo para una consulta con ejemplo

`SessionFactory`

- Factoría de sesiones.
- Su equivalente en JPA es `EntityManagerFactory`.

`Session`

- Representa una conexión a la BD a través de *Hibernate*.
- Su equivalente en JPA es `EntityManager`.

Métodos de `Session` I

Método	Uso
`save()`	Persiste un objeto
`persist()`	Persiste un objeto
`get()`	Recupera un objeto
`load()`	Recupera un objeto
`delete()`	Elimina un objeto persistente
`refresh()`	Re-lee el valor de un objeto persistente desde el origen de datos
`update()`	Actualiza el valor de un objeto en el almacén de datos
`clear()`	Limpia la sesión
`flush()`	Sincroniza el estado de la sesión con el almacén de datos
`contains()`	Chequea si un objeto está asociado a la sesión
`beginTransaction()`	Crea/arranca una transacción
`getTransaction()`	Obtiene la transacción asociada a la sesión
`close()`	Se cierra la sesión

Métodos de `Session` II

Método	Uso
`saveOrUpdate()`	Permite re-asociar (volver a pasar la estado persistente) a una entidad

Transacción

- Operaciones que se realizan como un bloque “todo o nada”.

Transaction

Método	Uso
`commit()`	Vuelca al origen de datos todas las modificaciones Implícitamente realiza un `flush()` de la sesión
`rollback()`	Deshace las modificaciones realizadas en la transacción
`isActive()`	Informa si la transacción está actica
`begin()`	Arranca la transacción

Colecciones de Hibernate

- *Hibernate* maneja internamente colecciones

Colección Hibernate	Orden	Equivalente Java
Bag	NO	List / ArrayList
Bag con ID	NO	List / ArrayList
List	SI	List / ArrayList
Set	NO	Set
Map	SI	Map

Atributos colección

- Se recomienda que:
  - la clase declarada sea la interfaz `Collection`.
  - la implementación sea una clase que permita indexar, como `ArrayList`.

Ej.
```
Collection<Empleado> listaEmpleados =
        new ArrayList<Empleado>();
```

Herencia I

- Existen diferentes estrategias para tratarla:
  - **Una sola tabla** (por defecto) [`SINGLE_TABLE`]: equivalente a las jerarquías en el E-R, cuando se agrupan todos los atributos en una tabla y aparece un campo discriminante para distinguir el subtipo.
  - **Una tabla por clase (sin normalizar)** [`JOINED`]: se mantiene una tabla para la super-clase (que contiene sus atributos) y tantas tablas como sub-clases (que contienen los atributos de la sub-clase más los heredados de la super-clase). No existe ninguna relación entre las tablas, más allá de que comparten el/los campo(s) identificador(es).

Herencia II

- **Una tabla por clase (normalizada)** [`TABLE_PER_CLASS`]: equivalente a las jerarquías en el E-R, cuando se mantiene una tabla para la super-entidad (que contiene todos los atributos comunes) y una tabla para toda sub-entidad (que contiene sólo los atributos específicos). La tabla de la super-clase y las de las sub-clases comparten el/los campo(s) identificador(es). En las tablas de las sub-clases, el identificador es clave primaria y ajena que apunta a la tabla de la super-clase.

Excepciones frecuentes

- En el paquete `org.hibernate`

Excepción	
`TransientObjectException`	Al relacionar un objeto persistido con otro no persistido (*Transient*)
`NonUniqueObjectException`	Al tratar de asociar a la sesión (haciendo `save()` o `update()`) un objeto con el mismo id que tiene un objeto ya asociado
`exception.ConstraintViolationException`	Se ha producido un error, a nivel de BD, de violación de alguna restricción

Consultas con nombre (*Named Queries*)

- Es una forma de etiquetar las consultas más frecuentes con un nombre mediante el cual se puedan referenciar.
- Se declaran en forma de anotaciones en los POJOs.
- Pueden ser en HQL o en SQL (nativas).

HQL

- *Hibernate Query Language*. Lenguaje de estándar de consultas de *Hibernate*.
- Es un subconjunto de JPQL.
- Sintácticamente es muy semejante a SQL, pero las consultas son sobre Clases (POJOs), no sobre Tablas.

Ej. FROM Empleado WHERE oficio = "Vendedor"

POJO Empleado.java

Atributo del POJO Empleado.java

HQL

- FROM Emp
- FROM Emp e WHERE e.empno
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- BETWEEN … AND …
- Recuperación ansiosa vs. perezosa
  - Ansiosa (*eager*): los objetos vinculados se inicializan

    ```
    FROM Cat AS cat INNER JOIN FETCH cat.mate LEFT
    JOIN FETCH cat.kittens
    ```
  - Perezosa (*lazy*): los objetos vinculados no se inicializan

    ```
    FROM Cat AS cat INNER JOIN cat.mate LEFT JOIN
    cat.kittens
    ```

Helpers

- Clases donde se implementan las funcionalidades específicas por entidades del problema. Generalmente se refiere a los DAO.

  Ej. EmpleadoHelper.java contendrá:

  - Empleado devuelveEmpleadoPorId(Integer idEmple)
  - List<Empleado> devuelveTodosEmpleados()
  - int eliminaEmpleado(Integer idEmple)
  - int creaEmpleado(Empleado e)
  - int modificaEmpleado(Integer idEmple, Empleado e)
  - …

Caché

- Hay varios niveles:
  - Primer nivel: Sesión
  - Segundo nivel:
    - Entre sesiones en una aplicación
    - Entre aplicaciones
    - Entre *clusters* de equipos

Caché de primer nivel – Sesión I

- Se mantiene la caché de sesión operativa mientras la sesión está abierta. Al cerrar la sesión se resetea la caché.
- Para mantener un objeto en caché entre sesiones, hay que recurrir a la caché de segundo nivel.

Caché de primer nivel – Sesión II

- *Hibernate* es capaz de detectar que un objeto ha sido “recientemente” recuperado y no lo vuelve a consultar.

  Ej: Sólo lanzará una SELECT a la BD, detecta que el segundo `get()` solicita un objeto recién recuperado:

```
sesion.beginTransaction();
Usuario us1 =
   (Usuario) sesion.get(Usuario.class, 1);
Usuario us2 =
   (Usuario) sesion.get(Usuario.class, 1);
sesion.getTransaction().commit();
```

ECLIPSE

Operativa

- Se instalará un *plug-in*, consistente en una serie de asistentes, que ayudará en la construcción de la capa intermedia que es *Hibernate*.
- Su manejo no es inmediato. Los asistentes de NetBeans son más fáciles de utilizar. Los asistentes de Eclipse son más útiles cuando ya se manejan.

Instalación

1. Eclipse (edición X)
2. En el Marketplace:
    1. Jboss Tools (edición X) 4.1.1 Final
        1. **Hibernate Tools**
        2. Jboss Maven Hibernate Configurator
        3. Jboss Maven Project Examples
        4. Jboss Tools Apache Tomcat Integration
        5. Jboss Tools Java Standard Tools
        6. Jboss Tools JSF
        7. Jboss Tools Visual Page Editor
        8. Jboss WebServices Tools

Configuración

- Menú Window → Open perspective → Hibernate

Añadir librerías al proyecto

- Se añaden las librerías externas necesarias
- Build path → Add external archives
  - Driver MySQL J/Connector
  - C:\Program Files\Eclipse_Kepler_jee\ plugins\ org.jboss.tools.hibernate4_0_3.7.1.Final-v20131205-0918-B107\lib**required**

Añadir un driver de BD

- Consiste en aportar el JAR del driver.

  Ej. mysql-connector-java-5.1.28-bin.jar
- Window → Preferences → Data management → Connectivity → Driver definitions

Añadir Configuración *Hibernate*

- Pestaña Hibernate Configurations → Add Configuration…
  - Project: seleccionar el proyecto actual
  - Database connection: crear una nueva o usar una existente
  - Configuration file: crear uno nuevo o usar uno existente

Generación de código Hibernate

- Botón Hibernate → Hibernate Code Generation Configurations

Referencias I

- HIBERNATE - Relational Persistence for Idiomatic Java

  https://docs.jboss.org/hibernate/orm/4.3/manual/en-US/html/
- JSR 338 – Java Persistence API 2.1

  https://jcp.org/en/jsr/detail?id=338
- JPA vs. Hibernate

  http://www.genbetadev.com/frameworks/jpa-vs-hibernate

Referencias II

- Tutorials Point

  http://www.tutorialspoint.com/hibernate/index.htm
- Instalación de Hibernate Tools en Eclipse:

  http://www.mkyong.com/hibernate/how-to-install-hibernate-tools-in-eclipse-ide/
- Generación de código en Eclipse:

  http://www.mkyong.com/hibernate/how-to-generate-code-with-hibernate-tools/

Referencias III

- Annotations:

  https://docs.jboss.org/hibernate/stable/annotations/reference/en/html_single/

- Consultas por criterios:

  https://docs.jboss.org/hibernate/core/3.3/reference/es-ES/html/querycriteria.html

- Using Criteria in Hibernate for Advanced Queries:

  http://www.developer.com/db/using-criteria-in-hibernate-for-advanced-queries.html

Referencias IV

- **Hibernate Tools Reference Guide**:

  http://docs.jboss.org/tools/4.0.0.Final/en/hibernatetools/html_single/index.html

- Tutorial Hibernate – HQL:

  http://www.javatutoriales.com/2009/09/hibernate-parte-7-hql-primera-parte.html

- Hibernate 4 + JPA 2.1 @NamedStoredProcedureQuery Example

  http://howtodoinjava.com/jpa/jpa-21-namedstoredprocedurequery-annotation-example/

Referencias V

- Ejemplo de JPA , Introducción (I)

  http://www.arquitecturajava.com/ejemplo-de-jpa/